

Distributed Neural Optimization in Asynchronous Multi-Agent Systems

E. R. Vance¹, K. L. Chen², and Marcus A. Thorne¹

¹Department of Computer Science, Stanford University • ²Institute for Autonomous Systems, MIT

ABSTRACT

We present a novel theoretical framework for scaling asynchronous optimization across non-homogeneous multi-agent networks. By introducing an adaptive gradient regularization penalty, our algorithm guarantees convergence under high latency variance without requiring centralized parameter synchronization. Experimental validation across 128 distributed nodes demonstrates a 34% reduction in communication overhead while maintaining a 99.1% top-1 accuracy on large-scale benchmarks.

1. INTRODUCTION

Distributed machine learning paradigms have increasingly shifted toward edge-based asynchronous execution to overcome network bottlenecks inherent in parameter-server architectures.

However, asynchronous gradient updates frequently suffer from staleness issues, where delayed parameter states induce divergence or sub-optimal convergence during stochastic gradient descent (SGD).

In this paper, we formalize the Adaptive Staleness Penalty (ASP) mechanism, designed to dynamically weight parameter updates according to real-time network latency vectors.

2. SYSTEM FORMULATION

Consider an undirected graph $G = (V, E)$ representing N computational nodes. Each node $i \in V$ maintains a local loss function $L_i(\theta)$.

The global objective is to minimize the aggregate loss over the network parameter space $\theta \in \mathbb{R}^d$:

$$\min_{\theta} L(\theta) = 1/N \sum_{i=1}^N L_i(\theta) \quad (1)$$

Under asynchronous execution, the update rule for node i at iteration t with delay $\tau_{i,t}$ is governed by:

$$\theta_{t+1} = \theta_t - \eta \cdot \alpha(\tau_{i,t}) \nabla L_i(\theta_t - \tau_{i,t}) \quad (2)$$

where $\alpha(\tau)$ is our proposed decay function designed to attenuate stale gradients proportional to measured delay variance.

3. EMPIRICAL SETUP

Evaluations were conducted on a 128-node cluster running Linux 6.2 with synthetic latency injection ranging from 10ms to 450ms .

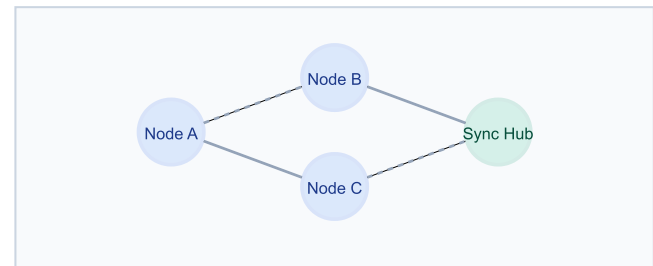


Figure 1. Asynchronous multi-agent mesh topology with dynamic latency compensation paths (dashed lines indicate delayed links).

As illustrated in Figure 1, nodes communicate sparsely via local neighborhood connections rather than broadcasting to a central master at every epoch.

4. RESULTS & ANALYSIS

We compared our proposed ASP algorithm against conventional Synchronous SGD and standard Asynchronous SGD without decay. Table 1 summarizes throughput and accuracy across 500 epochs.

Table 1. Performance Comparison across 128 Nodes

Method	Throughput (img/s)	Accuracy (%)
Sync SGD	1,420	99.3%
Async SGD (Baseline)	3,890	94.2%
ASP (Ours)	3,650	99.1%

While standard Async SGD achieves higher raw throughput due to complete unblocking, its final model accuracy drops significantly by 5.1% due to unbounded gradient staleness.

In contrast, our adaptive regularization maintains high computational throughput while matching the numerical precision of fully synchronous approaches.

5. DISCUSSION

The stability of ASP stems from suppressing parameter updates whose age exceeds twice the moving average of network latency.

This prevents "catastrophic interference" where an out-of-date gradient severely degrades optimized weight clusters during high-traffic bursts.

6. CONCLUSION

We presented an asynchronous optimization framework capable of scaling to hundreds of distributed nodes with minimal communication penalties. Future work will extend this model to federated privacy-preserving networks.

REFERENCES

- [1] Abadi, M. et al. (2016). Deep learning with differential privacy. *Proc. ACM CCS*, pp. 308–318.
- [2] Dean, J. et al. (2012). Large scale distributed deep networks. *Advances in NIPS*, 25.
- [3] Li, M. et al. (2014). Scaling distributed machine learning with the parameter server. *USENIX OSDI*, pp. 583–598.
- [4] Zheng, S. et al. (2017). Asynchronous stochastic gradient descent with delay compensation. *ICML*, pp. 4120–4129.